

Dokumentace k 2. projektu IPK - IPK25 Chat Client

Autor: Jan Kalina (xkalinj00)

Předmět: IPK - Počítačové komunikace a sítě

Akademický rok: 2024/2025

Obsah

- [Dokumentace k 2. projektu IPK - IPK25 Chat Client](#)
 - [Obsah](#)
 - [1. Úvod](#)
 - [2. Teoretický základ a účel aplikace](#)
 - [2.1 UDP varianta - datagramový protokol](#)
 - [2.2 TCP varianta - textový protokol](#)
 - [3. Sestavení a spuštění programu](#)
 - [3.1 Sestavení programu pomocí Makefile](#)
 - [3.2 Spuštění programu](#)
 - [4. Přehled architektury a struktura projektu](#)
 - [4.1 Modul Arguments](#)
 - [4.2 Modul Client](#)
 - [4.2.1 Sumbodul ClientFSM - Specializované konečné automaty pro TCP a UDP variantu](#)
 - [4.2.2 Submodul ClientOutput - Zpracování výstupu klienta](#)
 - [4.2.3 Submodul CommandParser - Zpracování uživatelských příkazů](#)
 - [4.3 Modul Common](#)
 - [4.4 Modul Exceptions](#)
 - [4.5 Modul Facades](#)
 - [4.6 Modul Messaging](#)
 - [4.6.1 Submodul MessageBuilder](#)
 - [4.6.2 Submodul MessageParser](#)
 - [4.6.3 Submodul MessagingHandler](#)
 - [4.7 Modul Networking](#)
 - [4.7.1 Submodul TCPCommunicationHandler](#)
 - [4.7.2 Submodul UDPCommunicationHandler](#)
 - [4.8 Modul App](#)
 - [5. Testování a verifikace funkčnosti](#)
 - [5.1 Interaktivní testovací konverzace mezi více instancemi klienta](#)
 - [5.1.1 Testovací prostředí](#)
 - [5.1.2 Testovací scénář](#)
 - [5.2 Testování pomocí testů od Vlad6422](#)
 - [5.2.1 Testovací prostředí](#)
 - [5.2.2 Testovací scénář](#)
 - [6. Závěr](#)
 - [7. Bibliografie](#)
 - [8. Přílohy](#)
 - [8.1 Adresářový strom projektu](#)
 - [8.2 Výstup příkazu make help](#)
 - [8.3 Ukázka spuštění programu s parametrem -h pro výpis nápovědy](#)
 - [8.4 Ukázka použití příkazu /help pro zobrazení nápovědy v průběhu chatování](#)

1. Úvod

Tato dokumentace popisuje **IPK25 Chat Client**, aplikaci navrženou pro komunikaci se vzdáleným chat serverem pomocí protokolu **IPK25-CHAT**. Klient podporuje obě varianty transportní vrstvy - **TCP** i **UDP** (pouze **IPv4**) - a zajišťuje autentizaci uživatele, možnost změny kanálů v rámci daného serveru, odesílání a přijímání zpráv nebo dokonce změnu uživatelského (zobrazovaného) jména. V průběhu vývoje byla zohledněna požadovaná funkcionality vycházející z detailní specifikace a také odlišnosti jednotlivých variant transportních protokolů, jako je například potvrzování zpráv v **UDP** variantě, které je zu protokolu **TCP** zajištěno interně. Vedle odlišností transportních protokolů jsem dále kladl velký důraz na návrh implementace zadaného Mealeho automatu, textové a bajtové formáty odesílaných a přijímaných zpráv, jejich efektivní validaci a reinterpetaci a řadu dalších aspektů, které jsou popsány v následujících kapitolách. Před samotnou implementací jsem navíc věnoval značné množství času objektovému návrhu a architektuře

aplikace, abych mohl v na projekt daném časovém okně, co nejlépe dodržovat zásady *clean code* a *single responsibility principle*.

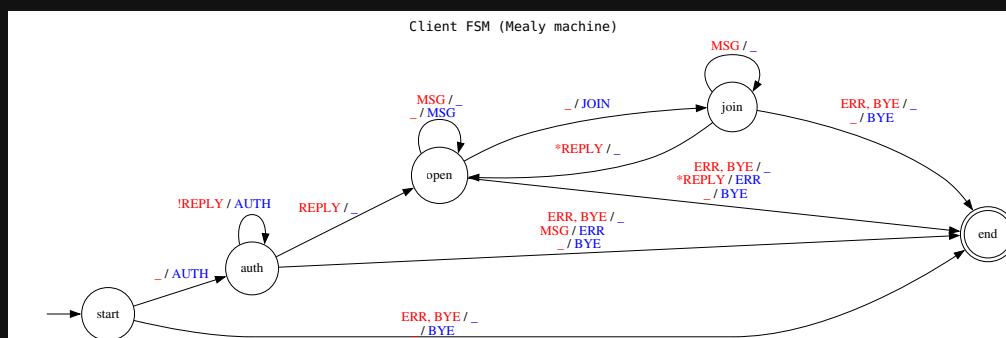
Z důvodu praktické a relativně nízkourovňové realizace využívá tento projekt jazyk **C++20** a je strukturován do několika modulů a dílčích submodulů, které jsou uspořádány v různých adresářích (viz [adresářový strom projektu](#)). Každý modul plní specifickou úlohu a dohromady vytvářejí architekturu, která dovoluje snadnou údržbu i rozšiřitelnost.

Celý vývoj projektu jsem verzoval pomocí systému *Git* na fakultním hostingu *Gitea*. Kompletní průběh vývoje si můžete snadno prohlédnout přímo v [repozitáři projektu](#). Pro lepší přehlednost jsem během práce důsledně používal *semantic commits*, což usnadňuje orientaci v jednotlivých změnách a pomáhá rychle pochopit jejich význam.

2. Teoretický základ a účel aplikace

Jak jsem již zmiňoval v úvodu, aplikace **IPK25 Chat Client** realizuje speciální protokol *IPK25-CHAT* [1], který definuje chování klienta a serveru v rámci chatovací služby pomocí konečného stavového automatu *Mealyho* typu [2]. Konečný stavový automat Mealyho typu (dále jen **FSM**) provádí přechody na základě externích událostí, vstupů, na jejichž základě provádí jasně definované akce, přechody, mezi jednotlivými stavy. V rámci přechodu může FSM provádět i další akce, jako například generování výstupních zpráv, které jsou zasílány uživateli – resp. může generovat výstupy. FSM Mealyho typu jsou proto vhodným nástrojem pro návrh a implementaci síťové komunikace.

Aplikace řídí celý životní cyklus komunikační relace – od navázání spojení přes autentizaci uživatele, přepínání mezi komunikačními kanály, odesílání a příjem zpráv až po korektní ukončení spojení, tzv. *graceful termination*. Implementace protokolu je realizována ve dvou odlišných transportních variantách (**TCP** a **UDP**), které se významně liší svým přístupem k zajištění spolehlivosti přenosu.



Obrázek 1: Konečný automat Mealyho typu uvedený v [zadáání projektu](#) [1]

2.1 UDP varianta - datagramový protokol

User Datagram Protocol (UDP), transportní protokol definovaný v **RFC 768** [3], je základní nespojovaný protokol transportní vrstvy, který poskytuje minimální služby pro přenos dat. Jeho hlavní charakteristikou je absence mechanismů pro zajištění spolehlivého přenosu, což znamená, že aplikace musí počítat s možností ztráty, duplikace či přeházení pořadí odeslaných paketů. Jak uvádí **RFC 5405 v sekci 3.1.1** [4], aplikace používající nespojový protokol **UDP** musí tolerovat ztrátu paketů, duplikaci a přeházené pořadí, což přesně odpovídá výzvam, se kterými jsem se potýkal při implementaci tohoto projektu.

V kontextu aplikace **IPK25 Chat Client** jsem musel implementovat vlastní mechanismy pro zajištění spolehlivosti, které nad základním **UDP** protokolem vytvářejí vrstvu poskytující funkcionalitu podobnou spojovaným protokolům (jako je **TCP**). Konkrétně jde o systém sekvenčního číslování zpráv, který umožňuje detekovat a řadit přijaté zprávy, mechanismus časových limitů a opakovaných přenosů pro případ ztráty datagramů, a také deduplikaci pro eliminaci duplicitně přijatých zpráv. Tyto mechanismy jsou v souladu s doporučeními **RFC 5405** [4], které popisuje osvědčené postupy pro aplikace využívající UDP.

2.2 TCP varianta - textový protokol

Transmission Control Protocol (TCP), transportní protokol specifikovaný v **RFC 793** [5], představuje robustní, spojově orientovaný protokol zajišťující spolehlivý přenos dat. Jak definuje **RFC 793 v sekci 1.5** [5], protokol **TCP** poskytuje spolehlivý a zabezpečený přenos dat mezi dvojicemi procesů.

Protokol **TCP** očekává, že z nižších vrstev komunikace může obdržet relativně nespolehlivé informace, proto bylo, dle mého názoru, mnohem pohodlnější zpracovávat **TCP** variantu projektu, protože veškeré nepříjemnosti spojené s používáním **UDP** protokolu, protokol **TCP** řeší sám a aplikační vrstva se tak může soustředit na zpracování zpráv a jejich formátování namísto řízení a kontroly samotného přenosu dat – dle **RFC 7414** [6], které mapuje vývoj **TCP** specifikací, protokol obsahuje pokročilé mechanismy pro řízení toku dat a předcházení zahlcení sítě, což přispívá k celkové robustnosti komunikace.

V rámci vývoje **TCP** varianty aplikace využívám textový formát zpráv s jasně definovanými oddělovači (přesně dle specifikace projektu), což umožňuje snadno zpracovat příchozí data a to dokonce i ve fragmentovaném stavu.

3. Sestavení a spuštění programu

3.1 Sestavení programu pomocí Makefile

Pro sestavení aplikace je k dispozici robustní Makefile, který nabízí celou řadu příkazů usnadňujících práci s projektem. Makefile podporuje dva režimy: režim **pro odevzdání** a režim **pro vývoj**, mezi kterými lze jednoduše přepínat pomocí příkazů `make developer-mode` a `make submission-mode`. Verze pro odevzdání je minimalistická a slouží primárně k sestavení finálního binárního souboru aplikace. Naproti tomu režim pro vývoj obsahuje rozšířené funkce, jako jsou stavba projektu pomocí CMake, zabalení projektu a kontrola základních požadavků na archiv a jeho obsah, instalace závislostí a další užitečné funkce usnadňující vývoj.

```
make && ./ipk25chat-client -h
```

Makefile zahrnuje základní příkazy jako: `- make all` – sestaví finální aplikaci (binární soubor). `- make build` – sestavení aplikace pomocí CMake ve verzi pro vývoj nebo pomocí Make ve verzi pro odevzdání. `- make clean` – provede úklid generovaných souborů, rozdílný pro verzi vývojovou a odevzdávací. `- make doc` – vygeneruje dokumentaci projektu pomocí nástroje Doxygen do `doc/documentation.html`. `- make help` – zobrazí seznam a popis všech dostupných příkazů Makefile. `- make pack` – vytvoří ZIP archiv se soubory určenými pro odevzdání (pouze ve verzi pro vývoj).

Detailní popis všech dostupných příkazů naleznete v automaticky generované nápovědě mého sebedokumentujícího Makefile, kterou lze získat příkazem `make help` (viz příloha 8.2)

3.2 Spuštění programu

Program se spouští z příkazové řádky následujícím způsobem:

```
./ipk25chat-client [-h | --help] {-t udp0rTcp | --transport-protocol udp0rTcp} {-s ip0rHostname | --server ip0rHostname} [-p port | --port port] [-d timeout | --wait timeout] [-r max | --max-retransmissions max]
```

- `-h, --help`: Vypíše nápovědu a ukončí program s návratovým kódem 0.
- `-t, --transport-protocol`: Povinný parametr, který určuje použitý transportní protokol – buď hodnotu `tcp` pro spolehlivou, textovou komunikaci nad **TCP**, nebo `udp` pro odlehčenou variantu s potvrzováním na úrovni aplikace využívající stejnojmenný protokol **UDP**. Akceptovatelné jsou pouze řetězce “`tcp`” a “`udp`” (bez ohledu na velikost písmen).
- `-s, --server`: Povinný parametr, který určuje adresu chatovacího serveru, kterou lze zadat buď jako **IPv4** adresu nebo platný *hostname* (textovou adresu serveru). Klient na tuto adresu naváže spojení (**TCP**) nebo pošle své první datagramy (**UDP**).
- `-p, --port`: Volitelný parametr, který určuje číslo portu serveru v rozsahu (1, 65535). Pokud je číslo portu blíže nespecifikováno, využije se výchozí hodnota 4567.
- `-d, --wait`: Volitelný parametr určený pro variantu protokolu **UDP** (zadán u **TCP** bude ignorován). Časový limit v milisekundách, po jehož uplynutí klient považuje potvrzovací zprávu za ztracenou a pokouší se o opětovné odeslání (*retransmisi*). Musí se jednat o kladné celé číslo, implicitní nastavení je 250 ms.
- `-r, --max-retransmissions`: Volitelný parametr určený pro variantu protokolu **UDP** (zadán u **TCP** bude ignorován). Tento parametr určuje maximální počet opakování odeslání (*retransmisi*) též zprávy při, ke kterým dojde při neobdržení potvrzení přijetí této zprávy ze strany serveru. Hodnota musí být nezáporné celé číslo, výchozí hodnotou jsou 3 opakování.

Vedle základních krátkých forem parametrů je možné použít i jejich dlouhé varianty nad rámec zadání, které jsou popsány výše. Výpis programové nápovědy si můžete prohlédnout v příloze 8.3

4. Přehled architektury a struktura projektu

Oproti prvnímu projektu do IPK, architektura tohoto projektu je mnohem větší. Celý projekt čistá dohromady přibližně 100 zdrojových a hlavičkových souborů, což je důsledkem mého pokusu vytvořit kvalitní objektové orientovaná návrh. Zdrojový kód je organizován do několika složek reprezentujících moduly aplikace s jasně vymezenou odpovědností a případně do dalších submodulů.

Architektura aplikace je postavena na principu vrstvení odpovědností, kde nejvyšší vrstva (App) využívá fasádu pro přístup k funkcionalitě, zatímco nižší vrstvy implementují konkrétní chování pro různé transportní protokoly.

Snažil jsem se u každého instanciovatelného modulu vytvořit společně veřejné **rozhraní** [7] (*interface*), které definuje vnější chování daného modulu. Díky tomu mohou ostatní moduly, které daný modul využívají, komunikovat s tímto modulem bez nutnosti znalosti jeho vnitřní implementace nebo toho, zda komunikuje s variantou modulu určenou pro **TCP** nebo **UDP** – varianty modulů obou variant vystupují *na venek* stejným způsobem.

Společné privátní rozhraní jsem se snažil definovat v báзовých třídách, kterou jsou někdy, stejné jako všechna rozhraní, čistě virtuální, anebo dokonce samy implementují metody sdílené oběma variantami klienta (**TCP** i **UDP**). I tento krok značně přispěl k lepší modularizaci a deduplikaci kódu, což vede také k jeho lepší udržitelnosti. Bohužel se musím přiznat, že jsem nestihl dokončit úplně kompletní refaktoring zdrojových souborů v tomto duchu – zejména jde o opakující se části kódu kontrolující návratové hodnoty síťových funkcí. Přesto se ale domnívám, že jsem zásady *clean code* [8] a *single responsibility principle* [9] dodržel v dostatečné míře, aby byl kód přehledný a snadno udržitelný.

Níže je rychlý přehled jednotlivých modulů a submodulů a jejich účelu:

- App: **Hlavní vstupní bod aplikace (main.cpp)**.
- Arguments: **Modul** zpracovávající parametry příkazové řádky a validuje vstupní argumenty.
 - ArgumentParser: Třída pro validaci a parsování argumentů příkazové řádky pomocí knihovny CLI11 [10].
 - CLI11.hpp: Knihovna třetí strany kompatibilní s mojí projektovou licencí poskytující příjemné rozhraní k parsování argumentů příkazové řádky.
- Client: **Modul** zapouzdřující hlavní logiku chatovacího klienta a implementaci stavového automatu.
 - ClientFSM: **Submodul** realizující konečný *Mealyho* automatu definujícího stavy a přechody klienta.
 - ClientOutput: **Submodul** implementující formátovaný výstup do terminálu na základě přijatých zpráv a jejich obsahu.
 - CommandParser: **Submodul** implementující logiku pro zpracování uživatelských příkazů a validaci jejich korektní struktury.
- Common: **Modul** definující sdílené datové třídy a datové typy používané napříč aplikací.
 - ChatDataTypes: Definice vlastních typů používaných v aplikaci, jako je společný variantní typ `MessageContent` pro **TCP** i **UDP**.
 - CommandLineOptions: Datová třída pro uchování zpracovaných argumentů příkazové řádky.
 - ParsedMessage: Datová třída pro uchování zpracovaných zpráv a jejich obsahu v podobě vektoru textových polí jejich parametrů.
 - UserCommand: Datová třída pro uchování uživatelských příkazů a jejich argumentů.
- Constants: **Modul** konstant různých účelů využívaných napříč programem.
 - ClientInternalPhrases: Textové konstanty dílčích frází, které jsou následně skládány do chybových zpráv vypsanych uživateli.
 - ClientLimits: Číselné konstanty omezující chování klienta (max. délka parametrů, povolené intervaly porty, apod.).
 - ColorEscapeSequences: Definice barevných sekvencí pro terminálové výstupy.
 - DefaultCommandLineOptions: Výchozí hodnoty pro argumenty příkazové řádky.
 - ExceptionMessages: Standardizované textové řetězce sloužící jako hlavní zprávy vlastních výjimek.
 - MessageFields: Definice počtu a indexů textových polí ve třídě `ParsedMessage` pro jednotlivé typy zpráv obou variant protokolů.
 - MessageKeywords: Klíčová slova obsažená ve zprávách protokolu **TCP** (např. *IS*, *OK*, *NOK*, ...)
 - RandomNumberRanges: Rozsahy náhodných čísel pro generování hodnot portů.
 - RegexPatterns: Regulární výrazy pro validaci vstupů, formátu zpráv a adres.
- Enums: **Modul** výčtových typů různých účelů využívaných napříč programem.
 - Mapping: **Submodul** využívající šablonové funkce k poskytnutí jednotného rozhraní pro mapování hodnot enumů na jejich textové reprezentace a naopak.
 - ClientFSMStates: Stavový konečného automatu klienta v rámci modulu `ClientFSM`.
 - ClientInternalErrorMessages: Výčtový typ vyjadřující výčet typů chybových hlášek klienta určených pro uživatele (v *mapperu* se mapují na konkatenci dílčích frází).
 - ExitCodes: Standardizované návratové kódy aplikace pro různé typy chyb.
 - MessageParameters: Výčtový typ pro jednotlivé parametry zpráv protokolu **IPK25-CHAT**.
 - MessageTypes: Výčtový typ pro jednotlivé typy zpráv, která navíc mají `uint8_t` hodnoty jejich **UDP kódu**.
 - TransportProtocolTypes: Výčtový typ pro rozlišení **TCP** a **UDP** protokolů.
 - UserCommandTypes: Výčtový typ pro rozlišení jednotlivých uživatelských příkazů.
 - ValidatorResults: Výčtový typ pro výsledky validačních funkcí (v pořádku, nevalidní znaky, nutné zkrátit, ...).
- Exceptions: **Modul** definující hierarchii výjimek pro různé typy chybových stavů.
 - ChatBaseException: Básová třída vlastních výjimek definující jejich obsah – stručnou zprávu, detail, chybový kód a kód chybové hlášky pro uživatele.
 - ChatExceptions: Specializované třídy výjimek – `InternalErrorException`, `ConnectionErrorException`, `UserInterruptedException`, ...
- Facades: **Modul** fasád, které zapouzdřují komplexní interakce a poskytuje zjednodušené rozhraní.
 - MainClientFacade: Centrální komponenta řídící tok aplikace a propojující ostatní části.
- Messaging: **Modul** implementující zpracování, validaci a parsování zpráv protokolu.
 - MessageBuilder: **Submodul**, který staví požadované zprávy s danými parametry v textovém formátu pro **TCP** a binárním pro **UDP** protokol.
 - MessageParser: **Submodul** sloužící pro převod textových/binárních dat na strukturované zprávy.
 - MessagingHandler: **Submodul** poskytující mezivrstvu mezi modulem `ClientFSM` a síťovým modulem `Networking`.

- **Networking: Modul** realizující síťovou komunikaci a abstrahuje rozdíly mezi protokoly.
 - `CommunicationHandlerBase`: Slouží zejména k zapouzdření společných třídních členů děděných do specializovaných variant.
 - `TCPCommunicationHandler`: Implementace spolehlivé komunikace pomocí TCP - využívá `connect()`, `send()` a `recv()`.
 - `UDPCommunicationHandler`: Implementace komunikace pomocí UDP včetně správy timeoutů, deduplikace a opakovaných přenosů - využívá `sendto()` a `recvfrom()` a dynamickou správu portů.
- **Utilities: Modul** poskytující doplňkové třídy obsahující pomocné metody využívané napříč celým projektem.
 - `CastUtils`: Šablonové funkce pro konverzi mezi různými typy dat (např. `uint8_t <-> EnumType`, `std::string <-> EnumType` a `size_t <-> IntegerType`, ...).
 - `CommunicationUtils`: Poskytuje metody pro rezoluci předané **IPv4** adresy nebo doménové adresy na jejich **IPv4** adresy.
 - `ExceptionHandler`: Centrální zpracování výjimek aplikace s volitelným výpisem chybových hlášek mapováním na návratové kódy a ukončením aplikace.
 - `Logger`: Jednoduché logovací makro pro flexibilní logovací výpisy, ladění a trasování běhu dat v aplikaci (**nasdílel jsem ho pull requestem do zadání**).
 - `RandomNumberGenerator`: Generátor náhodných čísel pro generování náhodných hodnot portů (*nakonec nebyl zapotřebí, ale v řešení ho ponechám, když už jsem ho vytvořil :-)*).
 - `StringUtils`: Pomocné funkce pro práci s řetězci, escapování a formátování.
 - `SignalHandler`: Zpracování signálů operačního systému (SIGINT, SIGSEGV).
- **Validators: Modul** implementující validační logiku pro dílčí parametry zpráv a zpráv jako celku v parsované podobě.
 - `MessageParametersValidator`: Slouží k validaci jednotlivých parametrů zpráv a jejich obsahu, dlouhé parametry zkracuje.
 - `MessageValidator`: Kontroluje přesně danou strukturu polí třídy `ParsedMessage` a jejich obsah (specificky pro **TCP** a **UDP**).
- *A další dílčí soubory – rozhraní, básové třídy, specializované třídy, ...*

Pro případné zájemce o implementační detaily doporučuji vygenerovat **Doxygen** dokumentaci pomocí příkazu `make doc` a prozkoumat ji (bude umístěna v `doc/documentation.html`). Obsahuje podrobnosti o třídách, metodách a jejich vzájemných vztazích.



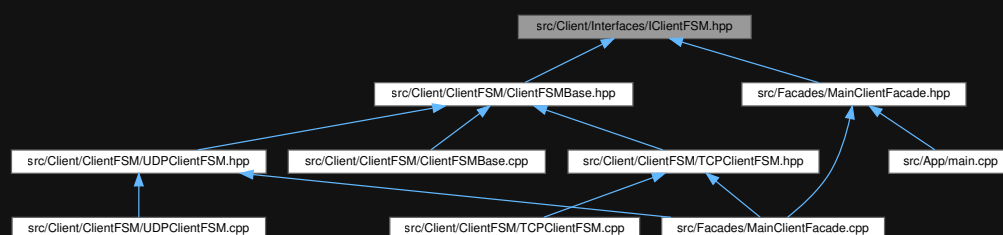
Obrázek 2: Graf volání z funkce `main()`

4.1 Modul Arguments

Parametry získané z příkazové řádky jsou uchovávány v instanci třídy `CommandLineOptions`.

Zpracování a validace vstupních argumentů probíhá prostřednictvím třídy `ArgumentParser`, která využívá knihovnu **CLI11** [10] pro parsování příkazové řádky. Tuto knihovnu jsem zvolil pro její jednoduchost a přehledný manuál, díky kterému je možné rychle se s knihovnou naučit pracovat. Současně je vedena pod **BSD licencí**, která je kompatibilní s **GNU GPLv3 licencí** mé aplikace.

4.2 Modul Client



Obrázek 3: Závislosti konečného automatu (FSM)

4.2.1 Sumbodul ClientFSM - Specializované konečné automaty pro TCP a UDP variantu

Jádrem celého modulu Client je implementace *FSM Mealyho typu*, který řídí chování klienta podle specifikace protokolu **IPK25-CHAT**. FSM definuje pět základních stavů: START (počáteční stav), AUTH (po odeslání autentizačních údajů), JOIN (žádost o připojení do jiného kanálu) a OPEN (stav, ve kterém klient přijímá zprávy) a END (ukončující stav sloužící k ukončení nekonečné běhové symčky FSM while(true)). Implementace obou verzí automatů závisí na opakovaném volání funkce poll(), která zajišťuje, že klient neustále čeká na události a reaguje na ně.

Implementace FSM se liší mezi **TCP** a **UDP** variantami zejména v oblasti zpracování přechodů mezi stavy. U **TCP** implementace je přechod stavů relativně přímočarý, protože protokol zajišťuje spolehlivé doručení zpráv ve správném pořadí. Naproti tomu **UDP** implementace musí řešit komplexnější scénáře, jako je retransmit odeslané BYE zprávy, opakování čekání na CONFIRM v modulu Networking, či řízení časovače čekání na očekávanou odpověď serveru REPLY. Stavový automat je navržen tak, aby v UDP variantě dokázal elegantně zpracovat potenciálně ztracené nebo zpožděné zprávy a udržet klienta v konzistentním stavu.

Na rozdíl od **TCP** varianty je u varianty **UDP** implementován buffer uživatelských příkazů, jelikož je potřeba, aby FSM nejdříve zpracovávalo příchozí zprávy, jejich potvrzování a případné retransmity vlastních zpráv a až poté je možné provádět další uživatelské příkazy. Pro jsem zavedl buffer těchto příkazů, aby uživatel mohl zadávat příkazy i v případě, že klient čeká na odpověď serveru.



Obrázek 4: Dědičnost tříd implementující konečný automat (FSM)

4.2.2 Submodul ClientOutput - Zpracování výstupu klienta

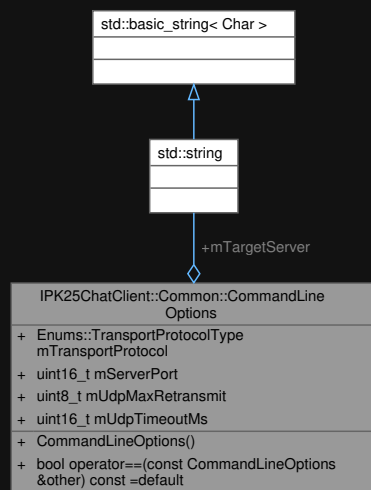
Submodul ClientOutput zajišťuje konzistentní a uživatelsky přívětivé zobrazování všech výstupů chatovacího klienta (dle zadání). Tato třída abstrahuje komplexitu formátování různých typů zpráv, jako jsou interní chyby klienta, chybová hlášení přijatá ze serveru, výpis nápovědy po zadání příkazu /help a přijaté zprávy od ostatních uživatelů či odpovědi serveru. Implementace používá různé

4.2.3 Submodul CommandParser – Zpracování uživatelských příkazů

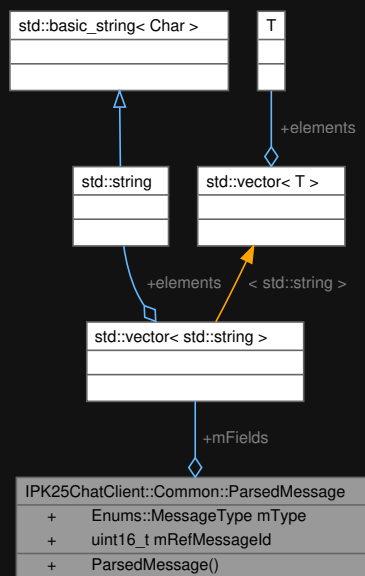
Třída `CommandParser` je zodpovědná za interpretaci uživatelských vstupů a jejich převod na interní příkazy. Implementace používá kombinaci regulárních výrazů (resp. validačních metod z modulu `Validators`) a stavového parsování pro identifikaci různých typů příkazů. `CommandParser` nejen rozpoznává příkazy, ale také validuje jejich formát a parametry. Například kontroluje maximální délku zpráv, povolené znaky v uživatelském jménu a další omezení stanovená protokolem. Výstupem tohoto modulu je inicializovaná instance třídy `UserCommand`, která obsahuje typ příkazu a jeho parametry.

4.3 Modul Common

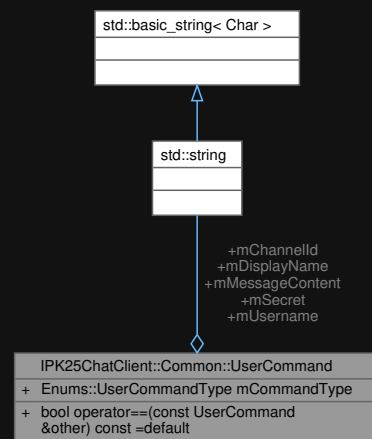
Tento modul obsahuje vedle jednoho variantního datového typu `MessageContent`, resp. `std::variant<std::string, std::vector<uint8_t>>` sloužícího k uchovávání textového obsahu zprávy pro **TCP** a binárního obsahu zprávy pro **UDP** již pouze datové třídy. Níže uvádím jejich diagramy tříd, ze kterých dle mého názoru bude mnohem lépe vidět, k čemu slouží a z jakých členů se skládají, než kdybych je popisoval slovy:



Obrázek 5: Třída `CommandLineOptions`



Obrázek 6: Třída `ParsedMessage`



Obrázek 7: Třída `UserCommand`

4.4 Modul Exceptions

Modul **Exceptions** poskytuje robustní mechanismus pro správu chybových stavů prostřednictvím výjimek. Základním stavebním kamenem tohoto modulu je třída **ChatBaseException**, která dědí od standardní třídy `std::exception`. Tato základní třída uchovává tři klíčové informace – chybový kód (reprezentovaný pomocí hodnoty výčtové třídy `ExitCode`), obecnou chybovou hlášku, volitelně dodatečný detail popisující vzniklou chybu a hodnotu výčtu `ClientInternalErrorMessages` rozhodující o tom, zda bude případně v rámci běhu FSM vytištěn do terminálu chybovou hlášku pro uživatele.

Na základě **ChatBaseException** byly díky dědičnosti vytvořeny specializované třídy výjimek. Mezi ně patří například **MessageLostErrorException** a **TimeoutErrorException**, které se vyvolají v **UDP** v případech, kdy vyprší 5vteřinový časový limit na přijetí odpovědi **REPLY** od serveru nebo při vypršení počtu možných retransmitů potvrzovací zprávy **CONFIRM** (v tom případě je zpráva ztracena). Dále modul obsahuje chybové výjimky jako **InvalidArgumentException**, **InternalErrorException** a další.

Chybové výjimky jsou zachytávány zejména v těle hlavní funkce FSM, kde se rozhoduje o tom, zda daná výjimka vyústí ve vypsání chybové zprávy pro uživatele, odeslání zprávy **ERR** nebo například, zda u **UDP** varianty dojde k odeslání zprávy **BYE** s maximálním počtem možných retransmitů. Dále jsou výjimky posílány dále, resp. do hlavní fasády uvnitř fasády `MainClientFacade`, kde je utility metoda `ExceptionHandler()` volány s příznakem pro ukončení programu (ve většině případech s chybovým návratovým kódem dle typu chycené výjimky). Aplikace díky tomu obsahuje robustní a konzistentní správu chybových situací.

4.5 Modul Facades

`MainClientFacade` je hlavní fasádou aplikace [11]. Je instanciována ve funkci `main()` a následně je pomocí jediného příkazu `appFacade.runChatClient(argc, argv)`; zahájen samotný běh aplikace. Jejím úkolem je:

- **zpracování příkazových argumentů** pomocí třídy `ArgumentParser`,
- inicializace instance třídy specializovaného `ClientFSM` pro daný transportní protokol dodržující veřejné rozhraní,
- spuštění běhu FSM pomocí volání jediné veřejné metody jejího rozhraní `mFsm->run();`,
- zachycení výjimek a jejich zpracování pomocí instance třídy `ExceptionHandler`.

4.6 Modul Messaging

4.6.1 Submodul MessageBuilder

Submodul MessageBuilder se stará o sestavování zpráv ve formátu požadovaném zvoleným transportním protokolem. Pro **TCP** implementaci vytváří textové zprávy s definovanou strukturou (např. MSG FROM username IS HelloWorld!), zatímco pro **UDP** sestavuje binární zprávy s pevnou hlavičkou obsahující **1B** typ zprávy, **2B** identifikátor zprávy a následně data specifická pro daný typ zprávy. Tato binární struktura je efektivnější pro přenos, ale vyžaduje přesné dodržení specifikované struktury a korektní zarovnání všech bajtů. Zároveň je nutné, abych při vytváření **UDP** zpráv k odeslání i validaci **UDP** příchozích zpráv korektně převáděl číselné hodnoty mezi 8bitovým a 16bitovým formátem a zároveň prováděl ve správnou chvíli konverzi mezi *Little Endian* a *Big Endian* formátem.

v **UDP** variantě tohoto submodulu je navíc integrována třída MessageIDProvider, která generuje a spravuje sekvenční čísla zpráv. Tato čísla jsou klíčová pro **UDP** implementaci, protože zajišťují unikátní identifikaci zpráv pro potřeby jejich potvrzování a případných opakovaných přenosů. **TCP** takový mechanismus nepotřebuje, protože spolehlivost přenosu je zajištěna samotným transportním protokolem. Třída MessageIDProvider má v sobě integrovaný zámek pro zajištění bezpečného přístupu k sekvenčním číslům a jejich inkrementaci (není to vysloveně nutné, jelikož moje aplikace je pouze jednovláknová, ale v brzkých fázích vývoje jsem si pohrával s myšlenkou vytvořit klienta vícevláknově – dle zadání by to ale bylo spíše ke škodě, než k užítku).

4.6.2 Submodul MessageParser

Submodul MessageParser řeší opačný proces než modul MessageBuilder, a to transformaci přijatých surových dat (textových nebo binárních) na strukturované zprávy reprezentované univerzální třídou ParsedMessage.

Implementace pro **TCP** protokol (TCPMessageParser) využívá pomocnou funkci ze submodulu StringUtils, která rozdělí přijatou textovou zprávu do fixního počtu polí na základě konkrétního typu přijaté zprávy. Následně probíhá validace, že ve vytvořených polích v rámci struktury ParsedMessage jsou skutečně očekávaná data pro daný typ zprávy. Parser dokáže díky internímu bufferu zpracovávat také fragmentované zprávy, jelikož platí, že každá zpráva je ukončena dvojicí znaků \r\n, díky čemuž lze zjistit, že se bufferu nachází nějaká celá zpráva, která je připravena k rozparsování.

UDP implementace (UDPMessageParser) naproti tomu pracuje s binárním formátem, kde každá zpráva má pevně danou strukturu s hlavičkou a datovou částí. Parser extrahuje typ zprávy, sekvenční ID a následně interpretuje obsah podle typu zprávy.

4.6.3 Submodul MessagingHandler

Submodul MessagingHandler představuje mezivrstvu mezi *FSM Client* a síťovou komunikací Networking. Dálo by se říci, že připravuje podklady pro odeslání síťovým modulem a post-procesuje síťovým modulem přijaté zprávy. Tento modul například převádí obsah datového typu MessageContent na požadovaný formát a inicializuje strukturu pro uživatelský příkaz UserCommand, která je přijímaná rozhraním submodulu MessageBuilder a vytváří tak zprávy pro odeslání. Tím se zjednodušuje celková složitost *FSM*, jelikož v rámci něj stačí zadat stačí ve správných místech volat metody pro odeslání konkrétní zprávy s předanými požadovanými parametry a submodul MessagingHandler s modulem Networking se postará o zbytek.

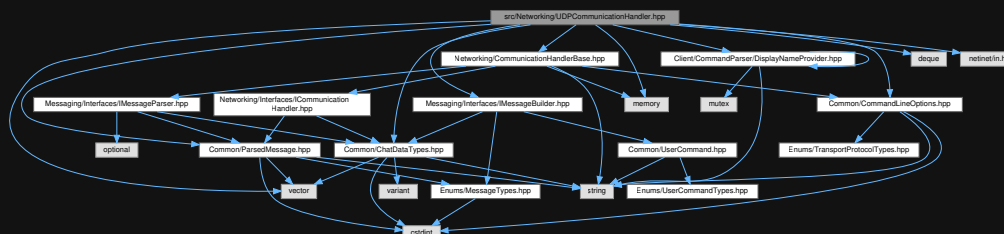
4.7.2 Submodul UDPCommunicationHandler

Submodul `UdpCommunicationHandler` musí řešit kritický problém nespolehlivé povahy **UDP** protokolu implementací vlastních mechanismů pro zajištění spolehlivého přenosu dat. Ačkoli je **UDP** bezstavový protokol, tak i zde má implementace vytváří socket a provádí rezoluci doménových jmen pomocí `getaddrinfo()` [17]. Díky tomu může klient pracovat jak s doménovými jmény, tak s **IPv4** adresami v textovém formátu.

Stěžejním prvkem implementace je sofistikovaný systém retransmisí zpráv. Po odeslání každé zprávy pomocí `sendto()` [18] metoda čeká na potvrzení této zprávy ze strany serveru pomocí zprávy `CONFIRM`. Pokud potvrzení není přijato v definovaném časovém limitu, zpráva je odeslána znovu, a tento proces se opakuje až do vyčerpání maximálního počtu pokusů. Příjem zpráv využívá neblokujícího volání funkcí `poll()` [16] a `recvfrom()` [19]. Současně je po příjmu zprávy kontrolováno, zda nedošlo k dynamické změně portu – v tomto případě se výslovně **UDP** třídní člen `mServerAddr.sin_port` nastaví na nový dynamický port. Zpracování dynamických portů serveru je velmi důležité při komunikaci přes NAT nebo proxy servery.

Deduplikaci zpráv zajišťují pomocí dvojice metod `markAsSeen()` a `wasSeen()` a fronty ID již viděných zpráv `mSeenIds`. Právě tyto dvě metody zajišťuje, že zprávy jsou zpracovány pouze jednou (viděné zprávy znovu nezpracovávám). Implementace této **UDP** varianty také mechanismus buferování zpráv ve vektoru `mStagedMessages` během čekání na potvrzení, takže žádná legitimní zpráva není ztracena.

Některé typy zpráv jsou zpracovávány přímo v submodule `UDPCommunicationHandler` a *FSM* je jimi vůbec nezabývá. Například zpráva typu `PING` slouží čistě k ověření, že je klient stále aktivní a tedy bohatě při jejím obdržení stačí přímo v rámci submodule `UDPCommunicationHandler` odeslat potvrzení o jejím obdržení. Zpráva `BYE` vyvolává speciální výjimku `ServerSendByeException`, která je ale vyhozena až poté, co submodule `UDPCommunicationHandler` přič zprávy `BYE` korektní potvrdí serveru. Následně je vyhozena zmiňovaná výjimka, která je zachycena v rámci *FSM* a dochází k úspěšnému ukončení běhu aplikace.



Obrázek 10: Třída `UDPCCommunicationHandler`

4.8 Modul App

Nakonec je zde modul App, který obsahuje vstupní bod aplikace v podobě souboru main.cpp. Jeho implementace je minimalistická – registruje signálové *handlers*, vytváří instanci třídy MainClientFacade. Dále invokes metodu runChatClient(argc, argv) čímž rozbíhá *kolesa* mého programu. Celý tento proces je zabalený do bloku try-catch – jde pouze *good-practice*, protože všechny výjimky by měly být zachyceny v rámci fasády (rád se ale držím hesla "*Better Sure Than Sorry*").

5. Testování a verifikace funkčnosti

5.1 Interaktivní testovací konverzace mezi více instancemi klienta

Tato sekce dokumentuje výsledky testování aplikace z hlediska její interaktivní schopnosti chatování pomocí **TCP** a **UDP** protokolů. Testování ověřovalo základní funkčnost mého chatovacího klienta, funkčnost všech příkazů a schopnost odesílat a přijímat zprávy. Následující testovací scénář postrádá pouze užití příkazu / help, jehož užití ale můžete v sekci 8.4.

Pro simulaci reálného provozu mého testovacího klienta jsem použil oficiální virtuální stroj s nainstalovaným vývojovým prostředím. Vývojové prostředí *IPK C* jsem spouštěl s administrátorským oprávněním uživatele **ipk**. Klient jako takový ovšem ke svému běhu superuživatelská práva nepotřebuje, takže jej lze spouštět i bez nich.

Pro efektivní testování komunikace mezi více klienty jsem vytvořil testovací skript `Server.py` (dostupný v adresáři `test/`), který simuluje chování vzdáleného chatovacího serveru na lokálním rozhraní. V zájmu časové efektivity jsem při tvorbě tohoto testovacího nástroje využil asistenci AI, konkrétně modelu *Claude 3.7 Sonnet Thinking* skrz rozšíření *GitHub Copilot*. Tento skript sloužil výhradně pro testovací účely, a proto jej nepovažuji za součást vlastního řešení projektu a vzdávám se veškerých autorských práv na tento skript a žádám, aby byl vyločen z kontroly plagiátorství a využití nástrojů **LLM** (pro jistotu ho přeci jen nechám dostupný pouze skrz *Gitea* repozitář a nebudu ho odevzdávat jako součást archvu do systému *StudIS*). Zdůrazňuji, že samotný chatovací klient, který je předmětem hodnocení, byl implementován zcela samostatně a případné využití generovaného či vypůjčeného kódu je řádně uvedeno u dotyčných metod v příslušných hlavičkových souborech.

Mezi hlavní schopnosti lokálního testovacího serveru patří: - podpora obou transportních protokolů a práce s příslušnou formou zpráv (textovou/binární), - korektní odesílání REPLY zpráv při autentizaci nebo změně kanálu, - zpracování všech příkazů (AUTH, JOIN, REPLY, BYE, ...) - implementace potvrzovacího mechanismu CONFIRM zpráv pro UDP komunikaci, - odesílání CONFIRM zpráv po přijetí zprávy od **UDP** klienta, - kontrola, zda klient potvrdil příjem zprávy ze serveru CONFIRM zprávou s korektním refMessageID, - možnost změny kanálů včetně upozornění o příchozech a odchodech uživatelů, - automatické generování a odesílání zpráv PING pro testování dostupnosti klienta.

Díky těmto vlastnostem server věrně napodobuje chování referenčního Discord serveru (anton5.fit.vutbr.cz) a poskytuje ideální prostředí pro testování chatovacího klienta v kontrolovaném prostředí bez nutnosti připojení k veřejnému serveru.

5.1.1 Testovací prostředí

- **Operační systém:** Ubuntu 24.04.2 LTS, amd64 (IPK25_Ubuntu24.ova)
- **Vývojové prostředí:** nix develop "git+https://git.fit.vutbr.cz/NESFIT/dev-envs.git?dir=ipk#c" [20]
- **Standard C++:** C++20
- **Překladač:** GCC 13.3.0
- **GNU Make:** verze 4.4.1
- **Knihovna CLI11:** verze 2.5.0

5.1.2 Testovací scénář

Testovací scénář byl navržen tak, aby pokryl všechny základní funkce chatovacího klienta. Jde o konverzaci mezi dvěma instancemi mého chatovacího klienta připojených k lokálnímu testovacímu serveru. Testovací scénář jsem navrhl jako rychlou konverzaci mezi třemi kamarády – *Johnem*, *Evou* a *Mikem*.

The image displays a Wireshark packet capture analysis. The top pane shows the packet list with details for selected packets. The bottom pane shows the packet bytes view.

Packet List:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	TCP	76	44472 → 4567 [SYN] Seq=309160495 Len=0 MSS=65536 Seq. PERN Total=3185188528 TSer=309160495
2	0.000000000	127.0.0.1	127.0.0.1	TCP	76	44472 → 4567 [ACK] Seq=309160495 Len=0 MSS=65536 Seq. PERN Total=3185188528 TSer=309160495
3	0.000000000	127.0.0.1	127.0.0.1	TCP	76	44472 → 4567 [ACK] Seq=309160495 Len=0 MSS=65536 Seq. PERN Total=3185188528 TSer=309160495
4	0.000000000	127.0.0.1	127.0.0.1	TCP	76	44472 → 4567 [ACK] Seq=309160495 Len=0 MSS=65536 Seq. PERN Total=3185188528 TSer=309160495
5	0.000000000	127.0.0.1	127.0.0.1	TCP	76	44472 → 4567 [ACK] Seq=309160495 Len=0 MSS=65536 Seq. PERN Total=3185188528 TSer=309160495
6	0.000000000	127.0.0.1	127.0.0.1	TCP	76	44472 → 4567 [ACK] Seq=309160495 Len=0 MSS=65536 Seq. PERN Total=3185188528 TSer=309160495
7	0.000000000	127.0.0.1	127.0.0.1	TCP	76	44472 → 4567 [ACK] Seq=309160495 Len=0 MSS=65536 Seq. PERN Total=3185188528 TSer=309160495
8	0.000000000	127.0.0.1	127.0.0.1	TCP	76	44472 → 4567 [ACK] Seq=309160495 Len=0 MSS=65536 Seq. PERN Total=3185188528 TSer=309160495
9	0.000000000	127.0.0.1	127.0.0.1	TCP	76	44472 → 4567 [ACK] Seq=309160495 Len=0 MSS=65536 Seq. PERN Total=3185188528 TSer=309160495
10	0.000000000	127.0.0.1	127.0.0.1	TCP	76	44472 → 4567 [ACK] Seq=309160495 Len=0 MSS=65536 Seq. PERN Total=3185188528 TSer=309160495
11	0.000000000	127.0.0.1	127.0.0.1	TCP	76	44472 → 4567 [ACK] Seq=309160495 Len=0 MSS=65536 Seq. PERN Total=3185188528 TSer=309160495
12	0.000000000	127.0.0.1	127.0.0.1	TCP	76	44472 → 4567 [ACK] Seq=309160495 Len=0 MSS=65536 Seq. PERN Total=3185188528 TSer=309160495
13	0.000000000	127.0.0.1	127.0.0.1	TCP	76	44472 → 4567 [ACK] Seq=309160495 Len=0 MSS=65536 Seq. PERN Total=3185188528 TSer=309160495
14	0.000000000	127.0.0.1	127.0.0.1	TCP	76	44472 → 4567 [ACK] Seq=309160495 Len=0 MSS=65536 Seq. PERN Total=3185188528 TSer=309160495
15	0.000000000	127.0.0.1	127.0.0.1	TCP	76	44472 → 4567 [ACK] Seq=309160495 Len=0 MSS=65536 Seq. PERN Total=3185188528 TSer=309160495
16	0.000000000	127.0.0.1	127.0.0.1	TCP	76	44472 → 4567 [ACK] Seq=309160495 Len=0 MSS=65536 Seq. PERN Total=3185188528 TSer=309160495
17	0.000000000	127.0.0.1	127.0.0.1	TCP	76	44472 → 4567 [ACK] Seq=309160495 Len=0 MSS=65536 Seq. PERN Total=3185188528 TSer=309160495
18	0.000000000	127.0.0.1	127.0.0.1	TCP	76	44472 → 4567 [ACK] Seq=309160495 Len=0 MSS=65536 Seq. PERN Total=3185188528 TSer=309160495
19	0.000000000	127.0.0.1	127.0.0.1	TCP	76	44472 → 4567 [ACK] Seq=309160495 Len=0 MSS=65536 Seq. PERN Total=3185188528 TSer=309160495
20	0.000000000	127.0.0.1	127.0.0.1	TCP	76	44472 → 4567 [ACK] Seq=309160495 Len=0 MSS=65536 Seq. PERN Total=3185188528 TSer=309160495
21	0.000000000	127.0.0.1	127.0.0.1	TCP	76	44472 → 4567 [ACK] Seq=309160495 Len=0 MSS=65536 Seq. PERN Total=3185188528 TSer=309160495
22	0.000000000	127.0.0.1	127.0.0.1	TCP	76	44472 → 4567 [ACK] Seq=309160495 Len=0 MSS=65536 Seq. PERN Total=3185188528 TSer=309160495
23	0.000000000	127.0.0.1	127.0.0.1	TCP	76	44472 → 4567 [ACK] Seq=309160495 Len=0 MSS=65536 Seq. PERN Total=3185188528 TSer=309160495
24	0.000000000	127.0.0.1	127.0.0.1	TCP	76	44472 → 4567 [ACK] Seq=309160495 Len=0 MSS=65536 Seq. PERN Total=3185188528 TSer=309160495
25	0.000000000	127.0.0.1	127.0.0.1	TCP	76	44472 → 4567 [ACK] Seq=309160495 Len=0 MSS=65536 Seq. PERN Total=3185188528 TSer=309160495
26	0.000000000	127.0.0.1	127.0.0.1	TCP	76	44472 → 4567 [ACK] Seq=309160495 Len=0 MSS=65536 Seq. PERN Total=3185188528 TSer=309160495
27	0.000000000	127.0.0.1	127.0.0.1	TCP	76	44472 → 4567 [ACK] Seq=309160495 Len=0 MSS=65536 Seq. PERN Total=31

Obrázek 11: Ukázka úvodu interaktivní testovací konverzace v programu **Wireshark**

```
root@ipk25:/home/jspk/server# (ninx@nix-shell-env) root@ipk25:/home/jspk/server# python3 server.py
Connection (TCP, 127.0.0.1, 4472): New client connected
[Auth (TCP, 127.0.0.1, 4472): AUTH client 1 USING *** AS client 1 John
Action Success: Welcome! You are now in channel default
/nomine John
System: User client 1 John has joined the channel
/client 1 eve: Hello
Mike: Hi, eve!
System: User client 2 Eve has joined the channel
/client 2 eve: Nice to see you John.
Still waiting for Mike?
/client 2 eve: Mike
System: User client 3 Mike has joined the channel
/client 3 Mike: Hello guys, lets go to our private channel 'besties'.
System: User client 3 Mike has left the channel
/client 3 Mike:
/nomine Mike
System Success: Joined channel besties
System: User client 2 eve has joined the channel
/client 2 eve:
/nomine Mike
Mike: Nice to chat with you again.
Same here!
/nomine John
User: Sorry guys, I must leave by pressing CTRL+C.
System: User client 2 eve has left the channel
/client 2 eve:
Mike: Well, then I shall press CTRL+C.
System: User client 3 Mike has left the channel
/client 3 Mike:
On my small server I am not even in shutting down it. Guess I'll receive EYE from it and terminate with 0.
(ninx@nix-shell-env) root@ipk25:/home/jspk/client_1# touch 3009.txt
(ninx@nix-shell-env) root@ipk25:/home/jspk/client_1# !a
root@ipk25:/home/jspk/client_2# (ninx@nix-shell-env) root@ipk25:/home/jspk/client_2# ./ipk25chat-client -t udp -s localhost
/nomine client 2 youg client 1 Mike
Action Success: Welcome! You are now in channel default
/client 1 John: Hi, Eve!
/client 1 John: Still waiting for Mike?
Yes!
System: User client 3 Mike has joined the channel
/client 3 Mike: Hello guys, lets go to our private channel 'besties'.
System: User client 3 Mike has left the channel
System: User client 1 John has joined the channel
/client 1 John:
/nomine John
System Success: Joined channel besties
/nomine Eve
Mike: Nice to chat with you again.
John: Same here!
User: Sorry guys, I must leave by pressing CTRL+C.
(ninx@nix-shell-env) root@ipk25:/home/jspk/client_2# touch EVE.txt
(ninx@nix-shell-env) root@ipk25:/home/jspk/client_2# !a
```

Obrázek 12: Ukázka průběhu interaktivní testovací konverzace na referenčním stroji

Jak je patrné ze *screenshotu* jejich komunikace, *John* s *Evou* se k serveru připojili jako první. *John* k připojení využil transportní protokol **TCP**, zatímco *Eva* zvolila **UDP**. *Mike* se připojil jako poslední taktéž si zvolil **UDP** variantu, a to s parametrizací počtu pokusů o znovuodeslání zpráv při ztrátě zprávy na 5 pokusů a dobu mezi retransmisemi na 350 ms.

John se jako první připojil k serveru a úspěšně se autentizoval pomocí příkazu /auth. Následně se připojil k kanálu general, kde začal chatovat s *Evou*, která se připojila a autentizovala chvíli po něm. *John* s *Evou* si

spolu vyměnili při čekání na *Mika* několik zpráv, přičemž si z výpisu konverzace můžete povšimnout, že klient správně zpracovává příchozí zprávy ze serveru a zprávy se korektně zobrazují všem uživatelům v kanálu general. U **UDP** varianty je navíc, vidět, že klient správně pracuje jak s příchozími, tak odchozími zprávami typu CONFIRM a v souladu se zadáním reaguje na příchozí zprávy typu PING odesláním potvrzení.

Mike se připojil k serveru jako poslední a úspěšně se autentizoval pomocí příkazu /auth. Následně se připojil k kanálu k *Johnovi* a *Evě* do kanálu general. *Mike* se pak přepojil pomocí příkazu /join do kanálu besties, kam se za ním obratem připojil i *John* s *Evou*. Opět si můžete povšimnout, že v případě **UDP** klientů server nejdříve odešle potvrzení na příchozí požadavek o změně kanálu, následně odešle odpověď REPLY o tom, že se přepojení povedlo, a klient na odpověď serveru reaguje potvrzením CONFIRM naopak ze své strany. V případě **TCP** klienta je to obdobné, ale server neodesílá potvrzení o přijetí zprávy, ale pouze odpověď REPLY na požadavek o změnu kanálu, na kterou klient v případě protokolu **TCP** nijak nereaguje.

V další části konverzace si uživatelé upravili svá zobrazovaná jména příkazem /rename. Tato funkce umožňuje změnit jméno, pod kterým se zprávy zobrazují ostatním účastníkům. Díky tomu se zprávy od uživatelů v chatu nezobrazují pod jejich původními jmény (např. *client_3_mike*), ale pod změněnými jmény (např. *Mike*). Server po této změně automaticky používá nová zobrazovaná jména při distribuci zpráv ostatním připojeným klientům.

Bohužel, *Eva* musí rychle odejít ze skupinového chatu (zvolila si vypnutí klienta pomocí klávesové zkratky CTRL+D), což vyústí také v odchod *Mika* (stisknutím CTRL+C). Nakonec zůstane připojený pouze *John*, kterému ale ze strany serveru přijde (například z důvodu plánované údržby) zpráva BYE, na což *Johnův* klient reaguje ukončením spojení a následně také ukončením aplikace s návratovým kódem 0. Jak *Eva*, tak *Mike* svým odchodem ze serveru způsobili korektní odeslání zprávy BYE serveru, čímž ho informovali o svém odchodu, načež čekali na potvrzení přijetí těchto zpráv ze strany serveru, jelikož komunikovali pomocí protokolu **UDP**. Po přojetí potvrzení ze strany serveru oba jejich klienti korektně ukončili spojení a následně také aplikaci s návratovým kódem 0. Tím jejich komunikace skončila.

Věřím, že tento testovací scénář dostatečně pokrývá všechny základní funkce chatovacího klienta a demonstruje jeho schopnost interaktivně komunikovat jak pomocí protokolu **TCP**, tak i pomocí protokolu **UDP**. Test dle mého názoru dopadl úspěšně a vše fungovalo dle mých očekávání.

Níže příkládám textový záznam této konverzace. Zprávy a příkazy *Johna* jsou reprezentovaný **tyrkysovou** barvou, *Evy* růžovou barvou a *Mika* **červenou** barvou. Zprávy odeslané serverem jsou zobrazeny **oranžovou** barvou:

Chat z pohledu *Johna*:

```
(nix:nix-shell-env) root@ipk25:/home/ipk/client_3# ./ipk25chat-client -t tcp -s localhost
/auth client_1 abc client_1_john
Action Success: Welcome! You are now in channel default
System: User client_2_eve has joined the channel
client_2_eve: Hello
Hi, Eve!
client_2_eve: Nice to see you John.
Still waiting for Mike?
client_2_eve: Yes!
System: User client_3_Mike has joined the channel
client_3_Mike: Hello guys, lets go to our private channel 'besties'.
System: User client_3_Mike has left the channel
/join besties
Action Success: Joined channel besties
System: User client_2_eve has joined the channel
/rename John
Mike: Nice to chat with you again.
Same here!
Eve: Sorry guys, I must leave by pressing CTRL+D.
System: User client_2_eve has left the channel
Mike: Well, then I shall press CTRL+C.
System: User client_3_Mike has left the channel
Oh no. I am all alone and the server is shutting down in 5s. Guess I'll receive BYE from it and terminate with 0.
```

Chat z pohledu *Evy*:

```
(nix:nix-shell-env) root@ipk25:/home/ipk/client_2# ./ipk25chat-client -t udp -s localhost
/auth client_2 xyz client_2_eve
Action Success: Welcome! You are now in channel default
Hello
client_1_john: Hi, Eve!
Nice to see you John.
client_1_john: Still waiting for Mike?
Yes!
System: User client_3_Mike has joined the channel
client_3_Mike: Hello guys, lets go to our private channel 'besties'.
System: User client_3_Mike has left the channel
System: User client_1_john has left the channel
/join besties
Action Success: Joined channel besties
/rename Eve
Mike: Nice to chat with you again.
```

```
John: Same here!  
Sorry guys, I must leave by pressing CTRL+D.
```

Chat z pohledu *Mika*:

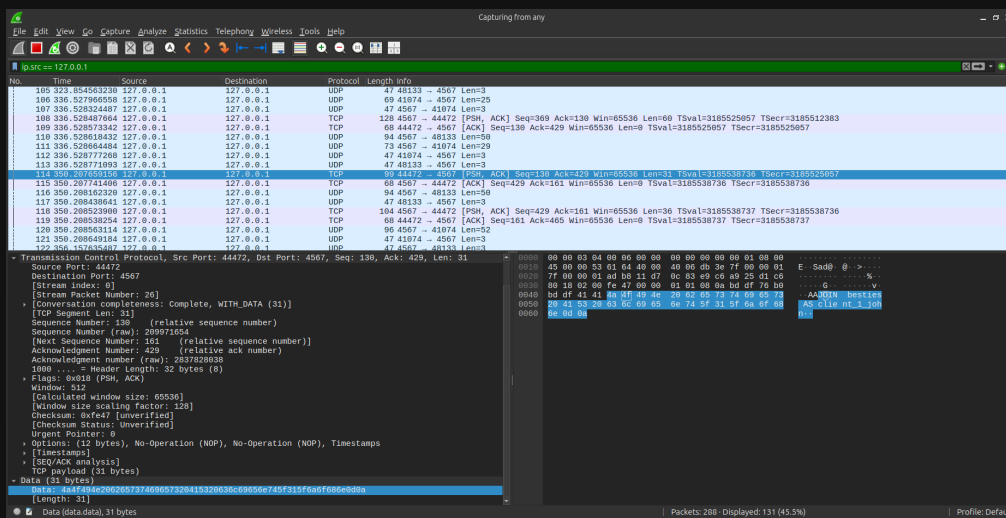
```
(nix:nix-shell-env) root@ipk25:/home/ipk/client_3# ./ipk25chat-client -s localhost -d 350 -r 5 -t udp  
/auth client_3 ipk client_3_Mike  
Action Success: Welcome! You are now in channel default  
Hello guys, lets go to our private channel 'besties'.  
/join besties  
Action Success: Joined channel besties  
System: User client_1_john has joined the channel  
System: User client_2_eve has joined the channel  
/rename Mike  
Nice to chat with you again.  
John: Same here!  
Eve: Sorry guys, I must leave by pressing CTRL+D.  
System: User client_2_eve has left the channel  
Well, then I shall press CTRL+C.
```

Chat z pohledu *Server* (rozesílání uživatelských zpráv není obsaženo):

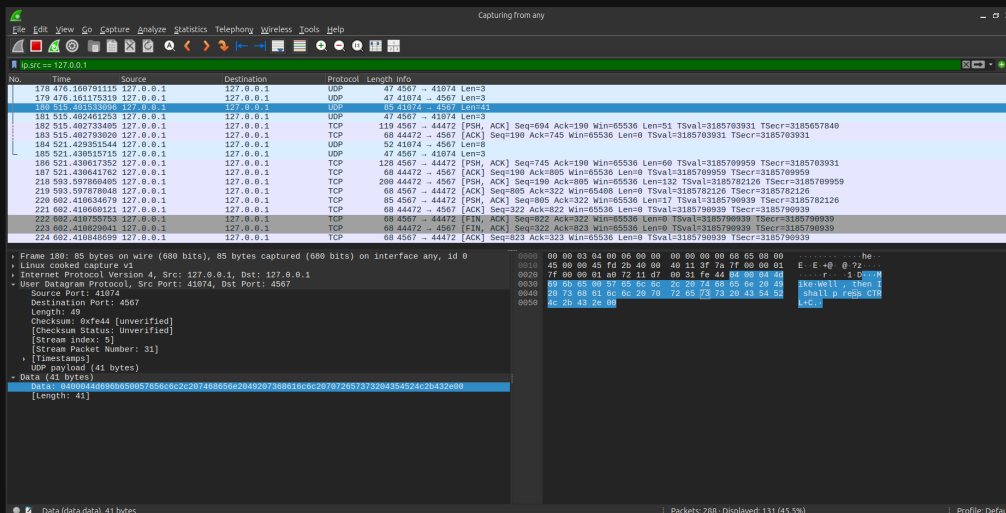
```
(nix:nix-shell-env) root@ipk25:/home/ipk/server# python3 server.py  
CONNECTION (TCP, '127.0.0.1', 44472): New client connected  
AUTH (TCP, '127.0.0.1', 44472): AUTH client_1 USING *** AS client_1_john  
SEND (TCP, '127.0.0.1', 44472): REPLY OK IS Welcome! You are now in channel default  
CONNECTION (UDP, '127.0.0.1', 48133): New client connected  
AUTH (UDP, '127.0.0.1', 48133): AUTH client_2 USING *** AS client_2_eve  
SEND (UDP, '127.0.0.1', 48133): CONFIRM 0 (0000)  
SEND (UDP, '127.0.0.1', 48133): REPLY OK IS Welcome! You are now in channel default  
CONFIRM (UDP, '127.0.0.1', 48133): REPLY 1 (0001) confirmed  
SEND (UDP, '127.0.0.1', 48133): PING 2  
CONFIRM (UDP, '127.0.0.1', 48133): PING 2 (0002) confirmed  
MSG (UDP, '127.0.0.1', 48133): MSG FROM client_2_eve IS Hello  
SEND (UDP, '127.0.0.1', 48133): CONFIRM 1 (0001)  
MSG (TCP, '127.0.0.1', 44472): MSG FROM client_1_john IS Hi, Eve!  
CONFIRM (UDP, '127.0.0.1', 48133): MSG 3 (0003) confirmed  
MSG (UDP, '127.0.0.1', 48133): MSG FROM client_2_eve IS Nice to see you John.  
SEND (UDP, '127.0.0.1', 48133): CONFIRM 2 (0002)  
MSG (TCP, '127.0.0.1', 44472): MSG FROM client_1_john IS Still waiting for Mike?  
CONFIRM (UDP, '127.0.0.1', 48133): MSG 4 (0004) confirmed  
SEND (UDP, '127.0.0.1', 48133): PING 5  
CONFIRM (UDP, '127.0.0.1', 48133): PING 5 (0005) confirmed  
MSG (UDP, '127.0.0.1', 48133): MSG FROM client_2_eve IS Yes!  
SEND (UDP, '127.0.0.1', 48133): CONFIRM 3 (0003)  
SEND (UDP, '127.0.0.1', 48133): PING 6  
CONFIRM (UDP, '127.0.0.1', 48133): PING 6 (0006) confirmed  
CONNECTION (UDP, '127.0.0.1', 41074): New client connected  
AUTH (UDP, '127.0.0.1', 41074): AUTH client_3 USING *** AS client_3_Mike  
SEND (UDP, '127.0.0.1', 41074): CONFIRM 0 (0000)  
SEND (UDP, '127.0.0.1', 41074): REPLY OK IS Welcome! You are now in channel default  
CONFIRM (UDP, '127.0.0.1', 41074): REPLY 8 (0008) confirmed  
CONFIRM (UDP, '127.0.0.1', 48133): MSG 7 (0007) confirmed  
SEND (UDP, '127.0.0.1', 48133): PING 9  
SEND (UDP, '127.0.0.1', 41074): PING 10  
CONFIRM (UDP, '127.0.0.1', 48133): PING 9 (0009) confirmed  
CONFIRM (UDP, '127.0.0.1', 41074): PING 10 (000A) confirmed  
MSG (UDP, '127.0.0.1', 41074): MSG FROM client_3_Mike IS Hello guys, lets go to our private channel 'besties'.  
SEND (UDP, '127.0.0.1', 41074): CONFIRM 1 (0001)  
CONFIRM (UDP, '127.0.0.1', 48133): MSG 11 (000B) confirmed  
JOIN (UDP, '127.0.0.1', 41074): JOIN besties AS client_3_Mike  
SEND (UDP, '127.0.0.1', 41074): CONFIRM 2 (0002)  
SEND (UDP, '127.0.0.1', 41074): REPLY OK IS Joined channel besties  
CONFIRM (UDP, '127.0.0.1', 41074): REPLY 13 (000D) confirmed  
JOIN (TCP, '127.0.0.1', 44472): JOIN besties AS client_1_john  
CONFIRM (UDP, '127.0.0.1', 48133): MSG 12 (000C) confirmed  
SEND (TCP, '127.0.0.1', 44472): REPLY OK IS Joined channel besties  
CONFIRM (UDP, '127.0.0.1', 48133): MSG 14 (000E) confirmed  
CONFIRM (UDP, '127.0.0.1', 41074): MSG 15 (000F) confirmed  
SEND (UDP, '127.0.0.1', 48133): PING 16  
SEND (UDP, '127.0.0.1', 41074): PING 17  
CONFIRM (UDP, '127.0.0.1', 48133): PING 16 (0010) confirmed  
CONFIRM (UDP, '127.0.0.1', 41074): PING 17 (0011) confirmed  
JOIN (UDP, '127.0.0.1', 48133): JOIN besties AS client_2_eve  
SEND (UDP, '127.0.0.1', 48133): CONFIRM 4 (0004)  
SEND (UDP, '127.0.0.1', 48133): REPLY OK IS Joined channel besties  
CONFIRM (UDP, '127.0.0.1', 48133): REPLY 18 (0012) confirmed  
CONFIRM (UDP, '127.0.0.1', 41074): MSG 19 (0013) confirmed  
MSG (UDP, '127.0.0.1', 41074): MSG FROM Mike IS Nice to chat with you again.  
SEND (UDP, '127.0.0.1', 41074): CONFIRM 3 (0003)  
CONFIRM (UDP, '127.0.0.1', 48133): MSG 20 (0014) confirmed  
SEND (UDP, '127.0.0.1', 48133): PING 21  
SEND (UDP, '127.0.0.1', 41074): PING 22  
CONFIRM (UDP, '127.0.0.1', 48133): PING 21 (0015) confirmed
```

```
CONFIRM (UDP, '127.0.0.1', 41074): PING 22 (0016) confirmed
MSG (TCP, '127.0.0.1', 44472): MSG FROM John IS Same here!
CONFIRM (UDP, '127.0.0.1', 48133): MSG 23 (0017) confirmed
CONFIRM (UDP, '127.0.0.1', 41074): MSG 24 (0018) confirmed
MSG (UDP, '127.0.0.1', 48133): MSG FROM Eve IS Sorry guys, I must leave by pressing CTRL+D.
SEND (UDP, '127.0.0.1', 48133): CONFIRM 5 (0005)
CONFIRM (UDP, '127.0.0.1', 41074): MSG 25 (0019) confirmed
BYE (UDP, '127.0.0.1', 48133): BYE FROM Eve
SEND (UDP, '127.0.0.1', 48133): CONFIRM 6 (0006)
CONFIRM (UDP, '127.0.0.1', 41074): MSG 26 (001A) confirmed
SEND (UDP, '127.0.0.1', 41074): PING 27
CONFIRM (UDP, '127.0.0.1', 41074): PING 27 (001B) confirmed
MSG (UDP, '127.0.0.1', 41074): MSG FROM Mike IS Well, then I shall press CTRL+C.
SEND (UDP, '127.0.0.1', 41074): CONFIRM 4 (0004)
BYE (UDP, '127.0.0.1', 41074): BYE FROM Mike
SEND (UDP, '127.0.0.1', 41074): CONFIRM 5 (0005)
MSG (TCP, '127.0.0.1', 44472): MSG FROM John IS Oh no. I am all alone and the server
is shutting down in 5s. Guess I'll receive BYE from it and terminate with 0.
^CSignal 2 received, exiting...
Stopping server...
SEND (TCP, '127.0.0.1', 44472): BYE FROM Server
```

Celou konverzaci jsem také zaznamenával pomocí programu Wireshark, ze kterého příkládám několik screenshotů demonstrující správný chod aplikace. Záznam této konverzace je dostupný také v adresáři test/ pod názvem test_communication_capture.pcapng.gzip.



Obrázek 13: Ukázka **TCP** zprávy JOIN zaslané klientem v programu **Wireshark**



Obrázek 14: Ukázka **UDP** zprávy MSG zaslané klientem v programu **Wireshark**

5.2 Testování pomocí testů od Vlad6422

5.2.1 Testování prostředí

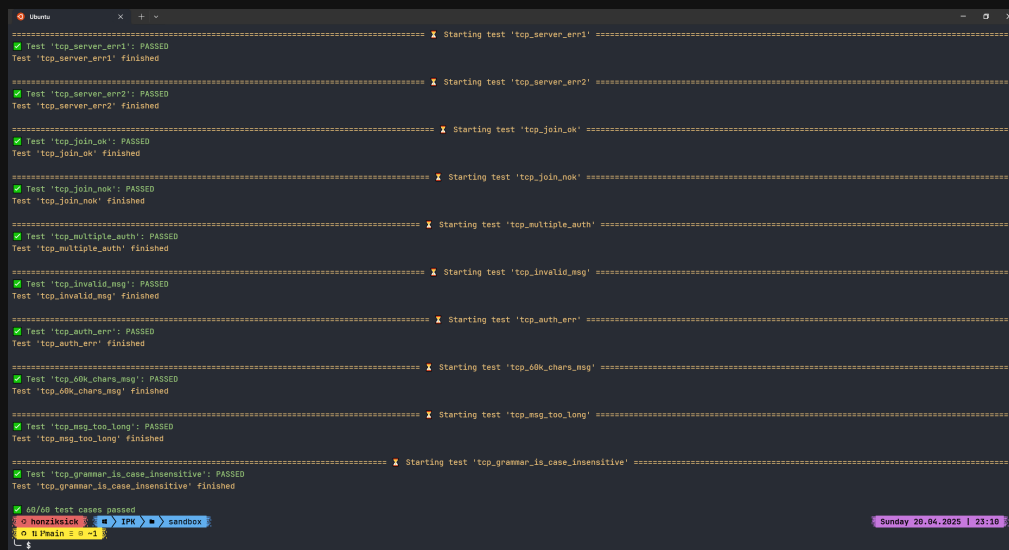
- **Operační systém:** Windows 11 WSL Ubuntu 24.04.2 LTS
- **Standard C++:** C++20
- **Překladač:** GCC 14.2.0
- **Knihovna CLI11:** verze 2.5.0

5.2.2 Testovací scénář

V rámci testování jsem použil také testy od *Vlad6422*, které jsou volně dostupné na portálu *GitHub* [21]. Testy jsem si mimo jiné doplnil i o své vlastní testy – např. testy pro korektní detekci příliš dlouhé messageContent anebo její přetečení a tedy nutnost ořezání, ...

Tyto testy byly velmi užitečné zejména ve fázi ladění a optimalizace protože díky nim šlo nasimulovat krátké testovací scénáře s možností přesně zadat očekávaný vstup i výstup klienta, nebo kontrolovat očekávané odchozí a příchozí zprávy včetně jejich obsahu a messageID.

Tímto bych tedy chtěl *Tomáši Hobzovi* alias *Vlad6422* velmi poděkovat za jeho krásné testy, které mi velmi pomohly při testování mé aplikace.



Obrázek 15: Výsledek testů od **Tomáše Hobzi** alias *Vlad6422*

6. Závěr

V tomto projektu jsem implementoval chatovacího klienta podle specifikace protokolu **IPK25-CHAT** s podporou pro dvě varianty transportního protokolu, **TCP** a **UDP**. Nejvýznamnější výzvou bylo vytvoření vlastního mechanismu spolehlivého přenosu pro **UDP** variantu, zahrnujícího číslování zpráv, retransmise, detekci duplicit a správu dynamických portů.

Podařilo se mi vytvořit modulární architekturu s jasnými rozhraními, která efektivně odděluje jednotlivé vrstvy aplikace a zapouzdřuje odlišnosti mezi protokoly. Díky tomu vyšší vrstvy aplikace pracují s oběma variantami transportních protokolů jednotným způsobem.

Během vývoje jsem se naučil mnoho o síťové komunikaci na nižší úrovni, zejména o praktických rozdílech mezi spojovanou a nespojovanou komunikací a o implementaci spolehlivého přenosu dat nad nespolehlivým protokolem. Přestože projekt obsahuje několik oblastí, které by zasloužily další refaktoring, považuji výslednou implementaci za robustní a dobře strukturovanou. Věřím, že mi získané zkušenosti s objektově orientovaným návrhem a implementací pomohou v budoucích projektech.

7. Bibliografie

- [1] **Dolejška, D.** *Projekt 2 – IPK25 Chat* [online]. VUT FIT Brno, únor 2025. Dostupné z: https://git.fit.vutbr.cz/NESFIT/IPK-Projects/src/branch/master/Project_2. [cit. 2025-04-20].
- [2] **Koziero, Ch. M.** *TCP Operational Overview and the TCP Finite State Machine (FSM)* [online]. TCPIPGuide, září 2005. Dostupné z: http://tcpipguide.com/free/t_TCPOperationalOverviewandtheTCPFiniteStateMachineF-2.htm. [cit. 2025-04-20].
- [3] **Postel, J** *RFC 768 – User Datagram Protocol* [online]. IETF, srpen 1980. Dostupné z: <https://datatracker.ietf.org/doc/html/rfc768>. [cit. 2025-04-20].
- [4] **Eggert, L., Fairhurst, G.** *RFC 5405 – Unicast UDP Usage Guidelines for Application Designers* [online]. IETF, listopad 2008. Dostupné z: <https://datatracker.ietf.org/doc/html/rfc5405>. [cit. 2025-04-20].
- [5] **Information Sciences Institute, University of Southern California** *RFC 793 – Transmission Control Protocol* [online]. IETF, září 1981. Dostupné z: <https://tools.ietf.org/html/rfc793>. [cit. 2025-04-20].
- [6] **Duke, M., Braden, B., Eddy, W., Blanton, E., Zimmermann, A.** *RFC 7414 – A Roadmap for Transmission Control Protocol (TCP)* [online]. IETF, únor 2015. Dostupné z: <https://datatracker.ietf.org/doc/html/rfc7414>. [cit. 2025-04-20].
- [7] **Křivka, Z.** *Principy objektově-orientovaného programování* [online]. VUT FIT Brno, březen 2025. Dostupné z: https://moodle.vut.cz/pluginfile.php/1064523/mod_folder/content/0/03-05-IPP_OO-Krivka-2025.pdf. [cit. 2025-04-20].
- [8] **Dybal, M.** *SOLIDní kód: Psaní čistého a udržitelného kódu* [online]. nesfit/ICS, březen 2019. Dostupné z: https://github.com/nesfit/ICS/blob/master/Lectures/Lecture_05/SOLIDni_kod.pdf. [cit. 2025-04-20].
- [9] **The GeeksForGeeks Community** *Single Responsibility in SOLID Design Principle* [online].

GeeksForGeeks, říjen 2023. Dostupné z: <https://www.geeksforgeeks.org/single-responsibility-in-solid-design-principle/>. [cit. 2025-04-20].

[10] **Schreiner, H.** *CLI11: An introduction* [online]. GitBook, leden 2025. Dostupné z: <https://cliutils.github.io/CLI11/book/>. [cit. 2025-04-20].

[11] **The Wikipedia Community** *Facade pattern* [online]. Wikipedia, leden 2025. Dostupné z: https://en.wikipedia.org/wiki/Facade_pattern. [cit. 2025-04-20].

[12] **Kerrisk M.** *connect(2) — Linux manual page* [online]. Linux man-pages project, červenec 2024. Dostupné z: <https://man7.org/linux/man-pages/man2/connect.2.html>. [cit. 2025-04-20].

[13] **Kerrisk M.** *send(2) — Linux manual page* [online]. Linux man-pages project, listopad 2024. Dostupné z: <https://man7.org/linux/man-pages/man2/send.2.html>. [cit. 2025-04-20].

[14] **Kerrisk M.** *recv(2) — Linux manual page* [online]. Linux man-pages project, listopad 2024. Dostupné z: <https://man7.org/linux/man-pages/man2/recv.2.html>. [cit. 2025-04-20].

[15] **Kerrisk M.** *shutdown(2) — Linux manual page* [online]. Linux man-pages project, červenec 2024. Dostupné z: <https://man7.org/linux/man-pages/man2/shutdown.2.html>. [cit. 2025-04-20].

[16] **Dolejška, D.** *Programování síťových aplikací: poll()* [online]. VUT FIT Brno, březen 2024. Dostupné z: https://moodle.vut.cz/pluginfile.php/1081875/mod_folder/content/0/IPK2024-25L-04-PROGRAMOVANI.pdf. [cit. 2025-04-20].

[17] **Kerrisk M.** *getaddrinfo(3) — Linux manual page* [online]. Linux man-pages project, listopad 2024. Dostupné z: <https://man7.org/linux/man-pages/man3/getaddrinfo.3.html>. [cit. 2025-04-20].

[18] **Kerrisk M.** *sendto(3p) — Linux manual page* [online]. Linux man-pages project, 2017. Dostupné z: <https://man7.org/linux/man-pages/man3/sendto.3p.html>. [cit. 2025-04-20].

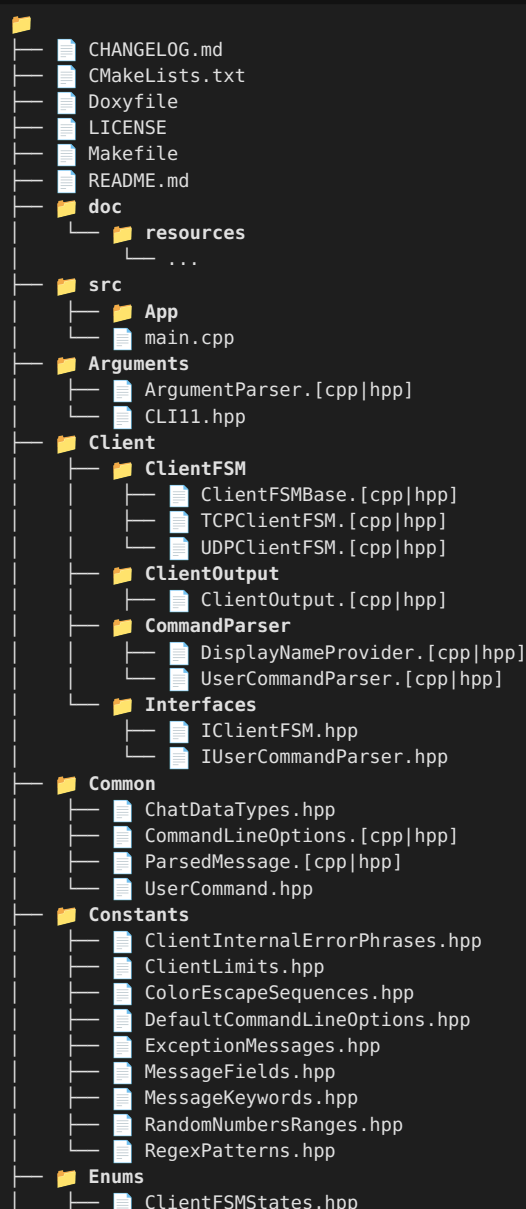
[19] **Kerrisk M.** *recvfrom(3p) — Linux manual page* [online]. Linux man-pages project, 2017. Dostupné z: <https://man7.org/linux/man-pages/man3/recvfrom.3p.html>. [cit. 2025-04-20].

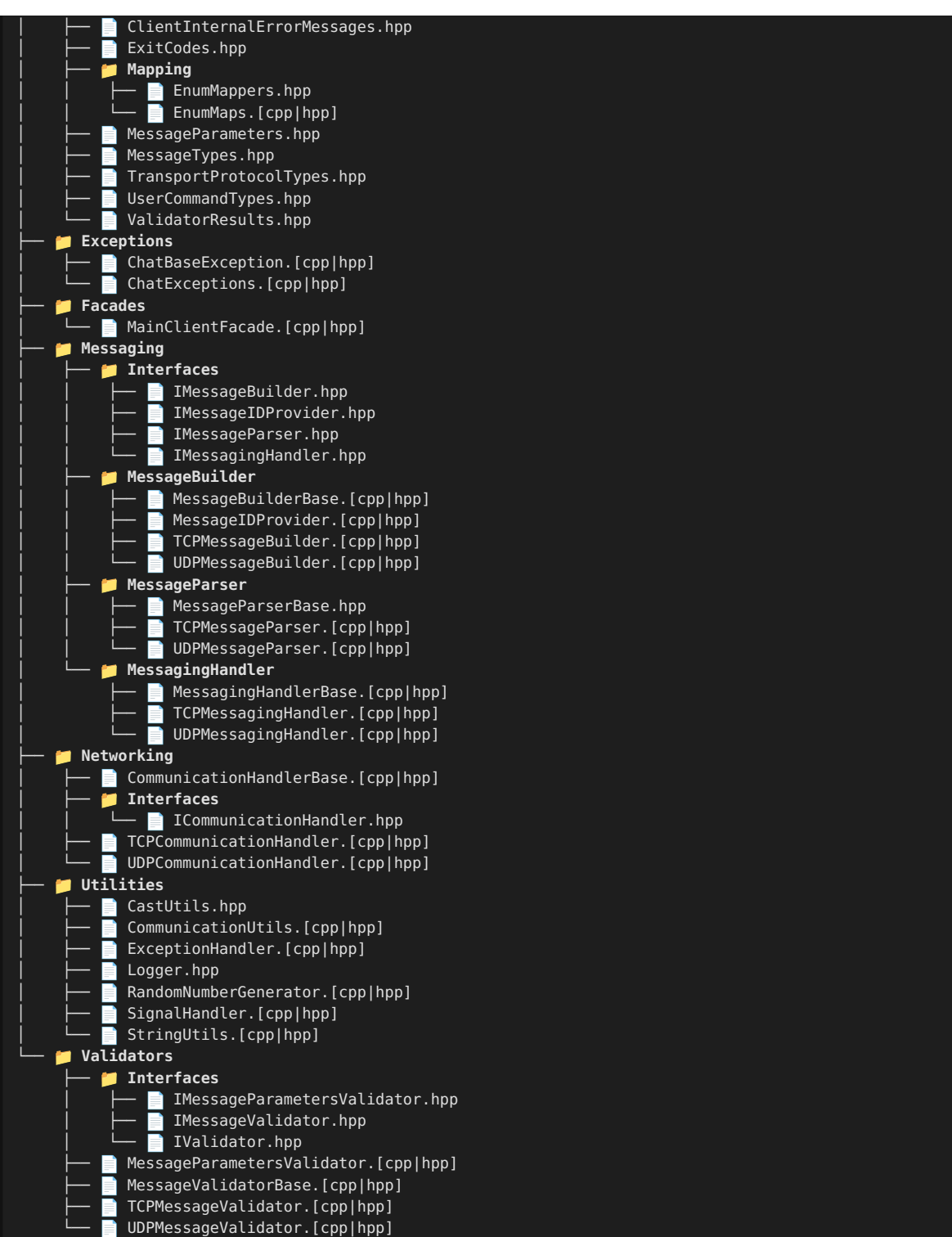
[20] **VUT FIT NESFIT tým** *Shared Development Environments* [online]. VUT FIT Brno, březen 2025. Dostupné z: <https://git.fit.vutbr.cz/NESFIT/dev-envs>. [cit. 2025-04-22].

[21] **Hobza, T., Malashchuk, V.** *VUT_IPK_CLIENT_TESTS* [online]. GitHub, duben 2025. Dostupné z: https://github.com/Vlad6422/VUT_IPK_CLIENT_TESTS/tree/main. [cit. 2025-04-22].

8. Přílohy

8.1 Adresářový strom projektu





8.2 Výstup příkazu make help

make help

Main Commands:

all	Builds the 'ipk25chat-client'
build	Builds the 'ipk25chat-client' via CMake in developer version and Make in submission version
clean	Runs 'clean-all' in developer / submission mode (different versions)
doc	Generates project documentation into the 'doc' directory (different versions)
help	Prints help for using the Makefile
pack	Creates a ZIP archive with files intended for submission (not allowed for submission)

Clean (special):

clean-all	Removes all created files (build, doc, executable, archive, ...)
clean-build	Removes the 'build' directory
clean-doc	Removes generated content of the 'doc' directory
clean-exec	Removes the executable
clean-pack	Removes the 'pack' directory including the archive (not allowed for submission)

Pack (special):

pack-prepare	Copies all necessary files to the 'pack/xkalinj00' directory (not allowed for submission)
--------------	---

Install Dependencies:

developer-mode	Switches the Makefile to developer mode
install-dev-dep	Installs dependencies needed for using all 'Makefile' functions (not allowed for submission)

<code>install-doc-dep</code>	Installs dependencies needed for generating documentation - 'doxygen' (not allowed for submission)
<code>install-help-dep</code>	Installs dependencies needed for printing 'Makefile' help - 'less' (not allowed for submission)
<code>install-pack-dep</code>	Installs dependencies needed for project packaging - 'rsync', 'zip' (not allowed for submission)
<code>submission-mode</code>	Switches the Makefile to submission mode
<code>update-dep</code>	Updates the list of available packages (not allowed for submission)

8.3 Ukázka spuštění programu s parametrem -h pro výpis nápovědy

```
./ipk25chat-client -h
```

IPK25 Chat Client v1.0

DESCRIPTION:

IPK25 Chat Client implements the IPK25-CHAT protocol over TCP or UDP (IPv4 only). Supports user authentication, channel management, message exchange and graceful termination. UDP variant includes application-level CONFIRM and retransmission logic; TCP variant uses a simple text grammar over a reliable stream.

USAGE:

```
./ipk25chat-client [-t udpOrTcp | --transport-protocol udpOrTcp] [-s ipOrHostname | --server ipOrHostname]
                  <-p port | --port port> <-d timeout | --wait timeout> <-r max | --max-retransmissions max>
```

OPTIONS:

-h,	--help	Display this help message and terminate the program with exit code 0
-t,	--transport-protocol	Transport protocol to use: "tcp" or "udp"
-s,	--server	Hostname or IPv4 address of the chat server
-p,	--port	Server port (default: 4567)
-r,	--max-retransmissions	UDP confirmation timeout in milliseconds (default: 250)
-d,	--wait	Maximum number of UDP retransmissions (default: 3)

EXAMPLE USAGE:

```
./ipk25chat-client -t tcp -s 127.0.0.1
./ipk25chat-client -t tcp -s localhost
./ipk25chat-client -t udp -s chat.example.com -p 10000
./ipk25chat-client -t udp -s 192.168.1.5 -p 3000 -d 100 -r 1
```

EXIT CODES:

```
0 - Success
64 - Invalid argument (usage error)
68 - Hostname resolution error
70 - Internal error
71 - Connection (socket) error
76 - Protocol error
78 - Unknown error
107 - Connection unexpectedly not established
110 - Message lost (retransmission limit exceeded)
116 - Connection timed out
```

8.4 Ukázka použití příkazu /help pro zobrazení nápovědy v průběhu chatování

Růžově zbarvené zprávy byly zadány uživatelem, zatímco tyrkysově zbarvené zprávy byly vypsány chatovacím klientem.

```
printf "Sending message without an authentication.\n/dontKnowWhatToDo\n/help\n" |
./ipk25chat-client -s anton5.fit.vutbr.cz -t tcp
```

Sending message without an authentication.

ERROR: Messages cannot be sent prior to authentication or without joining a chat channel.

/dontKnowWhatToDo

ERROR: Oops, it appears you've entered an invalid command. You may use the '/help' command to view all valid client commands along with their description, structure and usage examples.

/help

Usage:

/auth <username> <secret> <displayName>

Authenticate with the chat server using your account name, password (secret) and the display name you wish to use.

/join <channelId>

Join or switch to the specified channel. If you omit this, you stay in the default channel assigned after AUTH.

`/rename <newDisplayName>`

Change your display name for subsequent messages.

`/help`

Show this help message.

`/bye`

Exit the chat client gracefully (sends BYE to server and closes connection).

Chatting:

To send a message to the current channel, just type your text without a leading slash and press Enter. Messages longer than the protocol-allowed maximum will be truncated, and you will see a local warning. Maximum length of a message is 60000 characters and only these characters are allowed: Printable characters with space and new line (enter).

Parameter Limits:

- <username>: Max. length 20, allowed characters: [a-zA-Z0-9_-] (e.g., Abc_00-7)
- <channelId>: Max. length 20, allowed characters: [a-zA-Z0-9_-] (e.g., Abc_00-7)
- <secret>: Max. length 128, allowed characters: [a-zA-Z0-9_-] (e.g., Abc_00-7)
- <displayName>: Max. length 20, allowed characters: Printable characters (0x21-7E)
- MessageContent: Max. length 60000, allowed characters: Printable characters with space and line feed (0x0A, 0x20-7E)

Examples:

```
/help
/auth myNameIsJohn password John_TheIPKMaster00
/join general
/rename let-the-points-rain
Hello world!
```